Score-based Diffusion for Invariant Detection

1st Connor Lewis
jve9ur

I. Abstract

Daikon is widely considered the standard for computing program invariants for later mutant discovery. However, it is limited in its invariant detection with its process of iterating over variables with specific invariant formats. If an exact invariant format is not specified, then it is impossible for that invariant to be detected. To combat these issues, this paper proposes score-based diffusion models trained at different noising levels on function input and output combinations. Through this training, the score-based diffusion model learns the manifold of the functions runs. By conducting four experiments run on representative functions, this paper demonstrates that performing a Kolmogorov-Smirnov test at 10 noise levels comparing the distribution of Euclidean norms of scores output by a validation set and a test set successfully identifies all mutant sets while Daikon only finds 62.5%. Both methods were able to identify the correct mutated variable(s) when they were able to find the mutant, with the KS test method flagging the variable with the highest average score in the 99.5th percentile at each noise level. These experiments demonstrate that using score-based diffusion models as built in invariants for mutant and mutated variable identification surpasses Daikon when used at the function level.

II. Introduction

A. Problem

Program invariants are properties over execution variables that hold across all executions. These rules are especially useful for detecting faulty changes to code. For example, a correct implementation of a stack should always have the stack size increase by one when the push function is exercised. If a change to the code generates runs that violate invariants, then the change is likely faulty.

The detection and logging of invariants is useful to developers altering their code bases. When a change is introduced that violates a stored rule, the rule helps catch the faulty code before it is pushed. The rules also help developers in locating faults introduced by changes, as the rules include specific variables. The rules can also be helpful in understanding a given program by highlighting the relationships between variables.

B. Existing Solutions

Daikon [1] is the main existing solution for invariant detection. It works by processing a program's trace files using stored invariant templates across all combinations of selected variables. If a rule template passes for all given traces, then it is output as an invariant. Although there are default templates, more diverse templates can be added if one suspects particular invariants.

Although Daikon is a strong tool for invariant detection, it has many significant limitations. Most prominently, invariants can only be detected if a template is provided for them. Additionally, computation time and resources can be hefty if a program is testing lots of variables and lots of invariants. With these computation issues, one cannot solve the first issue of template inclusion by including lots of templates. These problems can prevent Daikon from capturing useful invariants of more complicated forms and lead to a less robust tool.

C. Key Insights

The proposed theory is that program execution variables exist on a defined manifold that can be learned by a score-based diffusion model. The score model having an understanding of the run variable manifold would in effect give it an understanding of the relationships and invariants between all variables. When plausible program execution variables are run through the model, their score output will have a small magnitude. This is because the variables already exist on the manifold and do not need to be denoised. However, when an execution with a mutation is included, the run variables will exist off the manifold and will out a larger score. The correlation between anomalous run variables and high score outputs can be used to detect any mutations. This relationship should also hold for specific variables, meaning the model can be used to determine which variables are related to the problematic run. Overall, while this strategy does not explicitly list out invariants, it effectively contains the invariants in its model weights by determining if a run exists on the manifold. The models can then be used to detect mutations even if the exact invariants cannot be derived from the model. The key advantage in this setup is being able to detect invariants of any form of any variables instead of having to iterate through invariant formats. This would be especially helpful for larger code bases with many variables and uniquely formatted invariants.

D. Contributions

The contributions for this paper are:

1) A novel application of score based diffusion models as program invariant detectors
2) A comparison across 4 functions of Daikon to score Euclidean norm threshold and KS test mutant detection methods

3) A demonstration of the superiority of KS test mutant detection followed by score threshold based variable identification versus all other methods

III. Approach

A. Overview

Input and output data was generated from four different functions and fed to Daikon for invariant generation. The data was then used to train a score-based diffusion model for each function to denoise the outputs at 10 different representative noising levels given the corresponding inputs. Multiple noising levels are needed as mutations could be fine grained or large scale. The 99th and 99.5th percentiles of the Euclidean norms of the validation set were stored for every noising level as different mutant detection thresholds. The test set and mutant sets then had their runs' norms compared to these thresholds and were flagged as mutants if greater than 1% or 0.5% of their runs were greater than their respective threshold. As an alternative method, the Kolmogorov-Smirnov test was ran comparing each test and mutant set Euclidean norms to the validation set's norms to determine if they were drawn from the same distribution.

B. Test Suite

Four functions were used as a representative test suite: StackAr Push, StackAr Pop, Projectile motion simulator, and a Triangle Classifier.

StackAr and its associated Push/Pop functions were used in the Daikon paper [1] and represent a baseline where Daikon performs very well. They serve to demonstrate that score-based diffusion models do not do poorly on what Daikon already does well on. Both functions receive and output the stack array and an integer representing the top of the stack. The projectile motion simulator and triangle classifier examples include nonlinear relationships and work to demonstrate areas where Daikon struggles and score-based diffusion should not. The projectile motion program will receive an initial velocity, an initial angle above the ground, and output the time spent traveling, the maximum height of the projectile, and the final position of the projectile. Metric units/formulas will be used. Air resistance will not be considered, and a constant ground level will be assumed. The triangle classifier will take in the lengths of the 3 sides and classify the triangle as equilateral, scalene, or isosceles, and right, acute, or obtuse.

C. In-Distribution Data

The stack input data for Push and Pop is generated in two steps. First, a random integer, n, is sampled from a uniform distribution to define the top of the stack within the 16 dimension stack array. The n value will be between 1 and 16 for Pop and between 0 and 15 for push. Then, n integers were generated from a uniform distribution between 1 and 100 to fill in the stack values. For the push function, another random number between 1 and 100 is generated for the value being pushed.

The projectile motion's initial velocity will be a float generated from a uniform distribution between 0 and 10 meters per second, and the angle over the horizontal will be a float between 0 and 180 degrees. The triangle data sides will be uniformly random floats between 1 and 100, with the data being evenly distributed among each feasible classification combination. This manual structuring of data is to ensure that triangles of higher rarity (right, equilateral, and isosceles) are represented.

There are 12,000 instances for push, pop, and projectile motion, with a train/val/test split of 10,000/1,000/1,000. There are 12,012 instances for the triangle examples in a 10,010/1,001/1,001 train/val/test split to enforce equal data among the seven possible triangle types.

The input and output data are normalized for every function except for the triangle output being one-hot encoded. This is to ensure a variable's score is not higher because its data exists on a larger scale.

D. Model Structure

All models are feed-forward with 2 layers of 2048 nodes and use the Adam optimizer and ReLU activation function. The model receives the noising level, the function input variables as conditioning, and the noised function output values during training. The model processes this information and outputs the score of the outputs'. The Pop, Push, and Projectile functions' noising values are a geometric progression from 0.01 to 1, and the triangle noising values are from 0.001 to 1 to help capture the thin right triangle manifold. The noising values are encoded. The noising levels are encoded by applying the natural logarithm [2] before being fed into the models. The model format was developed by modifying the code from Constrained Synthesis with Projected Diffusion Models [3]. Once a model is trained, the noised x value can be swapped out with the output of actual runs to return their associated scores and examine if they are off the manifold.

E. Mutation Data Generation

Four code mutants were generated for Push and for Pop, and 8 mutants were generated for Projectile Motion and for Triangle Classification. All mutant files were generated manually with a single bug. The input data to the mutant functions is identical to the test set to ensure consistent evaluation. If any mutant does not generate any different outputs, then a different mutant will be considered or the dataset will be examined to make sure the relevant data is included. The specific mutants generated are included in Table I, Table II, Table III, and Table IV.

TABLE I
Pop Mutants

Mutant	Description
1	Removed stack pointer decrement
2	Removed setting of popped value to 0
3	Increase stack pointer by 1 instead of decreasing
4	Set 1's value to 0 instead of input.ptr's value to 0

TABLE II
PUSH MUTANTS

Mutant	Description
1	Always set pointer to 1
2	Always push 1
3	Don't add pushed value
4	Always push the value to the first index

TABLE III
PROJECTILES MUTANTS

Mutant	Description
1	Remove degrees to radians conversion
2	Make g -9.81 instead of 9.81
3	Simplify g to 10 from 9.81
4	Flipped sin to cos in time
5	Flipped sin to cos in max height
6	Flipped sin to cos in final position
7	Changed v_0^2 to v_0 in final position
8	Changed v_0^2 to $v_0 \times 2$ in max height

F. Mutation Detection Using Thresholds

Two thresholds are determined at the 10 different noise levels using the 99th and 99.5th percentiles of the Euclidean-norm of the scores of the validation set. If a given run has a Euclidean-norm greater than a threshold, then it will be flagged as resulting from a mutant. The test set and the mutated test sets will be classified using these thresholds, where greater than 1% of examples being flagged as a mutant will result in a set being flagged as a mutated set for the 99th percentile threshold and greater than 0.5% being used for the 99.5th percentile example. Mutant variable identification is classified as successful if the variable related to the mutation has the highest individual score across the noise values in which the mutation is detected.

G. Mutation Detection Using the Kolmogorov-Smirnov Test

This evaluation method compares the distributions of the test and mutant datasets' Euclidean-norms using the Kolmogorov-Smirnov Test. The test identifies if two samples are extracted from two different probability distributions with a given p-score and may be more robust to false positives than the threshold test. The test will be ran across all 10 noising levels, with a set being flagged as a mutant if it is flagged by the test at any of the levels. The alpha value was set to 0.01 and was corrected to be $0.01/n = 0.01/10 = 0.001$ where n is the number of noising levels. This correction is needed due to the number of tests increasing the false positive rate. If a set is flagged as a mutant, then the threshold from before would still have to be used to flag the problematic individual variables.

TABLE IV
TRIANGLES MUTANTS

Mutant	Description
1	Changed epsilon for right triangle classification to 1e6 instead of 1e-6
2	Swapped isosceles with scalene classification
3	Changed right triangle to acute classification
4	Removed sorting of sides to determine hypotenuse
5	Multiplied side squares instead of adding in sum of squares
6	Swapped acute and obtuse classification
7	Multiplied sides by 2 instead of squaring in sum of squares
8	Assume sorted sides array is descending order instead of ascending order

H. Example

Let us consider the projectile motion function as an example. After data generation and training as previously described, one mutant for the code could be the simplification of Earth's gravitational constant from 9.81 to 9. All test set inputs would be run using this code to store their mutated outputs. Then, the scores of the mutated set output by the model would be compared to the detection thresholds of the validation set, and, if successful, the set will be classified as mutated. Additionally, the scores of the mutated set would be compared against the scores of the validation set using the KS test and be classified as mutant if the test flags them as being from a different distribution.

I. Limitations

The approach is limited in it restricting its input variables to a certain domain in which they would not necessarily be in real life. The approach also only considers of fixed input and output size. The end result is also limited in not having defined invariants, as those are not realistic to derive from model weights. This study also uses a limited number of mutants that do not necessarily reflect the set of possible mutants. The functions are also a lower complexity example that may be difficult to generalize to larger code-bases with functions called in many different orders. The computation time for training a score-based diffusion model may also be longer than Daikon's run time for certain setups. Although, in the score based diffusion model testing an infinite number of invariants, one could say the runtime to get the same testing out of Daikon would be infinite. The score-based diffusion model is also vulnerable to out of distribution data, as that would likely have a higher score output. However, this could be curbed by having a wide range of training data.

IV. EVALUATION

A. Research Question and Metrics

The research question is *Can score-based diffusion models trained on function input/output variables act as robust invariant detectors?* It is measured using the mutation set detection rates and mutated variable detection rates of the score-based diffusion compared to Daikon. Mutant set detection rates, variable detection rates, and false positives from the original test set will be reported across the 99th percentile threshold, 99.5th percentile threshold, and the KS test as a comparison between the two strategies and Daikon.

Precision, recall, and F1 scores are not included due to lack of relevance for this experiment. False positives are only possible on the in distribution dataset, for which false negatives

are not possible. False negatives are only possible on the mutated sets, where false positives are not possible. The two sets are evaluated separately, so precision, recall, and F1 would not be informative.

B. Dataset and Tools

All data was generated using personally created programs. Daikon was used to generate its invariants, and PyTorch was used for generating the model. SciPy was used for its KS test feature, and NumPy was used for its threshold calculation. Other imports were used, but the aforementioned are the most significant.

C. Competing Approaches

To summarize previous sections, the competing methods for mutant function identification through function input/output combinations are Daikon, Euclidean norm validation set threshold flagging, and the Kolmogorov-Smirnov comparing the distributions of Euclidean norms to the validation set. After a mutant has been flagged by Daikon, problem variable identification will be done directly through Daikon's explicit invariants. If the mutated variable is flagged by any violated variant, then it will be considered to have identified the problem variable. Variable identification for the threshold and KS tests will be done by examining the variable with the highest average score in each threshold. Each threshold will be compared against each other to examine what level is needed for the best results.

D. Results

TABLE V
THRESHOLD POP RUN FLAG RATES

Mutant Number	99th Percentile	99.5th Percentile
Clean	6.20%	3.90%
0	100.00%	100.00%
1	99.90%	99.80%
2	100.00%	100.00%
3	92.50%	92.50%

TABLE VI
THRESHOLD PUSH RUN FLAG RATES

Mutant Number	99th Percentile	99.5th Percentile
Clean	5.30%	2.60%
0	93.00%	93.00%
1	98.50%	97.90%
2	100.00%	99.70%
3	93.00%	93.00%

Tables V, VI, VII, VIII list the flag rate of the 99th and 99.5th thresholds from the validation set. Although all mutant sets except one have a higher flag rate than the test set across all thresholds, the method is too noisy and classified the test set as a mutated set in all scenarios. Using thresholds as a mutant detection method appears too noisy to be usable in practice. An arbitrary threshold could be used for stability, but that is not sound either.

TABLE VII
THRESHOLD PROJECTILE RUN FLAG RATES

Mutant Number	99th Percentile	99.5th Percentile
Clean	6.90%	5.10%
0	99.20%	99.20%
1	99.90%	99.70%
2	71.50%	69.00%
3	99.50%	99.30%
4	94.30%	93.80%
5	94.70%	94.30%
6	88.00%	87.20%
7	86.10%	84.90%

TABLE VIII
THRESHOLD TRIANGLE RUN FLAG RATES

Mutant Number	99th Percentile	99.5th Percentile
Clean	9.59%	4.00%
0	100.00%	100.00%
1	48.85%	38.56%
2	24.88%	18.58%
3	32.67%	27.47%
4	47.45%	41.76%
5	68.73%	63.74%
6	52.75%	47.95%
7	47.45%	41.76%

TABLE IX
DAIKON VS. KS TEST DETECTION FOR POP

Mutant	Daikon	Daikon Var ID	KS Test	KS Var ID
Clean	Non-Mutant	N/A	Non-Mutant	N/A
1	Mutant	True	Mutant	True
2	Mutant	True	Mutant	True
3	Mutant	True	Mutant	True
4	Mutant	True	Mutant	True

TABLE X
DAIKON VS. KS TEST DETECTION FOR PUSH

Mutant	Daikon	Daikon Var ID	KS Test	KS Var ID
Clean	Non-Mutant	N/A	Non-Mutant	N/A
1	Mutant	True	Mutant	True
2	Mutant	True	Mutant	True
3	Mutant	True	Mutant	True
4	Mutant	True	Mutant	True

TABLE XI
DAIKON VS. KS TEST DETECTION FOR PROJECTILE

Mutant	Daikon	Daikon Var ID	KS Test	KS Var ID
Clean	Non-Mutant	N/A	Non-Mutant	N/A
1	Mutant	True	Mutant	True
2	Non-Mutant	False	Mutant	True
3	Non-Mutant	False	Mutant	True
4	Mutant	True	Mutant	True
5	Mutant	True	Mutant	True
6	Mutant	True	Mutant	True
7	Non-Mutant	False	Mutant	True
8	Non-Mutant	False	Mutant	True

TABLE XII
DAIKON VS. KS TEST DETECTION FOR TRIANGLE

Mutant	Daikon	Daikon Var ID	KS Test	KS Var ID
Clean	Non-Mutant	N/A	Non-Mutant	N/A
1	Mutant	True	Mutant	True
2	Non-Mutant	False	Mutant	True
3	Non-Mutant	False	Mutant	True
4	Non-Mutant	False	Mutant	True
5	Non-Mutant	False	Mutant	True
6	Mutant	True	Mutant	True
7	Mutant	True	Mutant	True
8	Non-Mutant	False	Mutant	True

Using either method introduced zero false positives in identifying the test set as a mutant. Although Daikon succeeded on Push and Pop mutants, its overall identification performance was $15/24 = 62.5\%$. The KS test performed far better and had a rate of $24/24 = 100\%$. Both methods were able to identify the variable(s) influenced by the change if they were able to flag the code as a mutant. Daikon included the variable(s) in its explicit violated rules, and the KS and 99.5th percentile method made the problem variable have the highest score output on average. However, this high success rate could be due to the fact that some of the mutations impact many output variables, of which the identification of any would results in a success by the metric. This sharp contrast demonstrates the superiority of comparing score Euclidean norms using the KS test over Daikon.

V. CONCLUSIONS

A. Major Findings

This work found that score-based diffusion models maintain Daikon's strengths in mutant and relevant variable identification in areas where Daikon excels but extends the successes to nonlinear areas where Daikon fails. Score-based diffusion models combined with the KS test comparing score Euclidean norm distributions was able to identify mutants based on function input/output 100% of the time compared to Daikon's 62.5%. Both methods were able to identify the problematic variables when they were able to identify a mutant, making the variable identification the same rate as the mutant identification. This demonstrates that score-based diffusion combined with KS testing on score Euclidean norms and threshold variable identification serves as a robust alternative to Daikon.

When examining Daikon's invariants, it primarily struggled with nonlinear invariant formats as expected. Although it was able to find the classic comparator examples using $<, >, ==,$ and $!=$, it lacked the ultra-specific invariant formats required by the nonlinear programs. Although the selected programs have invariants that are well-known in general (such as how to find a right triangle and the projectile motion equations), these rules are not always going to be known. Using score-based diffusion to the unknown invariant cases should be much faster than Daikon due to its ability to capture all possible invariants in one model.

B. Current Unknowns

The main unknown of the new score-based diffusion method is how well it will scale to larger functions and code bases. In particular, how well they would scale to code bases with an unknown sequence of actions (such as different orders of pushes and pops). However, there should be confidence in its ability to handle these circumstances, due to its proven ability to handle other time sequence modalities such as video generation.

C. Later Works

Future testing would need to transform the current score-based diffusion architecture into one that handles time series data. Then, just like Daikon, it would be able to track changes across all variables at every computation step. If successful, the method would not only be able to detect the variable related to the mutant but the exact computation step that the run went off the manifold. This would tell developers the exact combination of variable/computation that put the run off the manifold, which would be extremely helpful in fault isolation.

REFERENCES

[1] M. D. Ernst, J. Cockrell, W. G. Griswold, and D. Notkin, "Dynamically discovering likely program invariants to support program evolution," *IEEE Transactions on Software Engineering*, vol. 27, no. 2, pp. 99–123, 2001.

[2] T. Karras, M. Aittala, T. Aila, and S. Laine, "Elucidating the Design Space of Diffusion-Based Generative Models," *Advances in Neural Information Processing Systems*, 2022.

[3] J. K. Christopher, S. Baek, and F. Fioretto, "Constrained Synthesis with Projected Diffusion Models," *Advances in Neural Information Processing Systems*, 2024.